

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

Утвержден на заседании кафедры

«Вычислительная техника»

" ____ " _____ 20 ____ г.

Заведующий кафедрой

_____ М.А. Митрохин

ОТЧЕТ ПО УЧЕБНОЙ (ОЗНАКОМИТЕЛЬНОЙ) ПРАКТИКЕ

(2025/2026 учебный год)

Лысов Илья Владимирович

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения _____ 1 _____ семестр _____ 2 _____

Период прохождения практики с 24.06.26 по 07.07.26

Кафедра «Вычислительная техника»

Заведующий кафедрой д.т.н., профессор, Митрохин М.А.

(должность, ученая степень, ученое звание, Ф.И.О.)

Руководитель практики д.т.н., профессор, Зинкин С.А.

(должность, ученая степень, ученое звание)

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

Утвержден на заседании кафедры

«Вычислительная техника»

" ____ " _____ 20 ____ г.

Заведующий кафедрой

_____ М.А. Митрохин

**ИНДИВИДУАЛЬНЫЙ ПЛАН ПРОХОЖДЕНИЯ УЧЕБНОЙ
(ОЗНАКОМИТЕЛЬНОЙ) ПРАКТИКИ**

(2025/2026 учебный год)

Лысов Илья Владимирович

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения _____ 1 _____ семестр _____ 2 _____

Период прохождения практики с 24.06.26 по 07.07.26

Кафедра «Вычислительная техника»

Заведующий кафедрой д.т.н., профессор, Митрохин М.А.

(должность, ученая степень, ученое звание, Ф.И.О.)

Руководитель практики д.т.н., профессор, Зинкин С.А.

(должность, ученая степень, ученое звание)

№ п/п	Планируемая форма работы во время практики	Количество часов	Календарные сроки проведения работы	Подпись руководителя практики от вуза
1	Выбор темы и разработка индивидуального плана проведения работ	2	24.06.2026 - 25.06.2026	
2	Подбор и изучение материала по теме работы	15	26.06.2026 – 28.06.26	
3	Разработка алгоритма	43	01.07.26 – 03.07.26	
4	Описание алгоритма и программы	18	03.07.26 – 04.07.26	
5	Тестирование	5	04.07.26 – 05.07.26	
6	Получение и анализ результатов	10	05.07.26 – 08.07.26	
7	Оформление отчёта	15	05.07.26 – 07.07.2026	
	Общий объём часов	108		

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ОТЧЁТ
О ПРОХОЖДЕНИИ УЧЕБНОЙ (ОЗНАКОМИТЕЛЬНОЙ) ПРАКТИКИ
(2024/2025 учебный год)

Лысов Илья Владимирович

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения 1 семестр 2

Период прохождения практики с 24.06.26 по 07.07.26

Кафедра «Вычислительная техника»

Лысов И. В. выполнял практическое задание «Сортировка пузырьком». На первоначальном этапе были изучен и проанализирован алгоритм пузырьковой сортировки, был выбран метод решения и язык программирования C, на котором было написано меню программы сортировки массива методом пузырька. Также, осуществил работу с файлами. Оформил отчёт.

Бакалавр Лысов И.В. " " 2026 г.

Руководитель Зинкин С.А. " " 2026 г.
практики

ОТЗЫВ
О ПРОХОЖДЕНИИ УЧЕБНОЙ (ОЗНАКОМИТЕЛЬНОЙ) ПРАКТИКИ
(2024/2025 учебный год)

Содержание

Введение.....	7
1. Постановка задачи.....	8
2. Выбор решения.....	10
3. Описание программы.....	11
4.Схема программ.....	14
4.1 Блок-схема программы	14
4.2 Блок-схема алгоритма	15
5.Тестирование программы	15
6. Отладка	16
7.Совместная работа	19
Заключение.....	22
Список используемой литературы.....	23
Приложение А. Листинг программы.....	24
Приложение Б. Результаты тестирования программы.....	31

Введение

В современных информационных системах обработка и упорядочивание данных являются важной частью работы программного обеспечения. Сортировка данных применяется в базах данных, информационных системах, поисковых системах и различных прикладных программах. Упорядоченные данные позволяют ускорить поиск информации, повысить эффективность обработки и упростить анализ результатов.

Существует множество алгоритмов сортировки, отличающихся сложностью реализации и скоростью работы. Одним из наиболее простых и наглядных алгоритмов является пузырьковая сортировка. Принцип работы данного алгоритма заключается в последовательном сравнении соседних элементов массива и их обмене в случае неправильного порядка. Процесс повторяется до тех пор, пока массив не будет полностью отсортирован. Несмотря на сравнительно низкую производительность при обработке больших объемов данных, данный алгоритм широко используется в учебных целях благодаря простоте реализации и наглядности работы.

В данной работе разработана программа на языке Си, предназначенная для загрузки числовых данных из текстовых файлов, выполнения пузырьковой сортировки и анализа эффективности алгоритма. Программа позволяет считывать данные из файлов форматов TXT и CSV, выполнять сортировку массива по возрастанию, определять количество сравнений и перестановок элементов, измерять время выполнения алгоритма и сохранять результаты обработки в файл.

Реализация программы позволяет изучить принципы работы с файлами, массивами, алгоритмами сортировки и средствами измерения производительности программ. Кроме того, в ходе выполнения работы были получены практические навыки коллективной разработки программного обеспечения с использованием системы контроля версий Git и сервиса GitHub.

1 Постановка задачи

Поставленная задача: разработать программу на языке Си, выполняющую загрузку массива числовых данных из текстового файла и его сортировку методом пузырька по возрастанию.

Программа должна обеспечивать возможность считывания данных из файлов форматов TXT и CSV с использованием различных разделителей, таких как пробел, запятая и точка с запятой. После загрузки данных необходимо выполнить сортировку массива, определить количество сравнений и перестановок элементов, а также измерить время выполнения алгоритма.

Результаты работы программы должны выводиться на экран пользователя. Кроме того, программа должна предоставлять возможность сохранения отсортированного массива в файл с поддержкой нескольких режимов записи: перезапись исходного файла, добавление данных в конец файла или сохранение результата в новый файл.

Для оценки эффективности алгоритма необходимо провести тестирование программы на различных наборах данных: случайном массиве, отсортированном массиве и массиве, упорядоченном по убыванию. Полученные результаты позволяют сравнить количество операций и время выполнения алгоритма в различных случаях.

Использовать сервис GitHub для совместной работы над проектом. Создать и разместить коммиты, отражающие вклад каждого участника бригады. Оформить отчет по выполненной работе.

Достоинства алгоритма пузырьковой сортировки:

- простота реализации алгоритма;
- наглядность работы алгоритма;
- возможность раннего завершения сортировки при отсутствии перестановок;
- удобство применения для небольших наборов данных;
- использование минимального объема дополнительной памяти.

Недостатки алгоритма пузырьковой сортировки:

- высокая алгоритмическая сложность $O(n^2)$;
- низкая производительность при обработке больших массивов данных;
- большое количество сравнений и перестановок;
- уступает современным алгоритмам сортировки по скорости работы.

Типичные сценарии применения данного алгоритма:

- учебные программы для изучения алгоритмов сортировки;
- сортировка небольших наборов данных;
- демонстрация принципов работы методов упорядочивания;
- обработка данных в задачах, не требующих высокой производительности;
- использование в качестве примера при изучении программирования и анализа алгоритмов.

2 Выбор решения

Для разработки программы нашей бригадой была выбрана среда разработки Microsoft Visual Studio и язык программирования Си.

Язык программирования Си является одним из наиболее распространенных языков программирования и широко применяется при изучении алгоритмов и структур данных. Данный язык обеспечивает высокую скорость выполнения программ, эффективную работу с памятью и предоставляет широкие возможности для реализации различных алгоритмов обработки данных. Благодаря своей простоте и универсальности язык Си позволяет наглядно реализовать алгоритм пузырьковой сортировки и организовать работу с файлами.

Microsoft Visual Studio представляет собой интегрированную среду разработки, предназначенную для создания консольных и графических приложений. Данная среда обладает удобным интерфейсом, средствами компиляции, отладки и тестирования программ, что значительно упрощает процесс разработки и поиска ошибок.

Для хранения входных и выходных данных в программе используются текстовые файлы форматов TXT и CSV. Программа поддерживает различные разделители данных, такие как пробел, запятая и точка с запятой. Использование файлов позволяет сохранять данные между запусками программы и проводить тестирование алгоритма на различных наборах входных данных.

Для совместной разработки программного продукта использовался сервис GitHub. Данная платформа предоставляет возможность хранения исходного кода, контроля версий и отслеживания изменений, внесенных участниками проекта. Использование GitHub и консольной утилиты Git Bash позволило организовать коллективную работу над программой, распределить задачи между участниками бригады и сохранить историю разработки проекта.

3 Описание программы

При запуске программы пользователю выводится главное меню, содержащее основные функции работы с массивом данных.

```
printf("1. Загрузить массив из файла\n");  
  
printf("2. Показать массив\n");  
  
printf("3. Сортировать\n");  
  
printf("4. Сохранить результат\n");  
  
printf("0. Выход\n");
```

Пользователю необходимо выбрать один из пунктов меню.

При выборе пункта «Загрузить массив из файла» открывается диалоговое окно выбора файла. Программа поддерживает загрузку данных из текстовых файлов форматов TXT и CSV.

```
if (DialogOpenFile(path))  
{  
    LoadFromFile(path);  
}
```

После открытия файла выполняется чтение его содержимого и разбиение строки на отдельные числовые значения. В качестве разделителей могут использоваться пробелы, запятые, точки с запятой и символы перевода строки.

```
token = strtok(text, ",; \r\n\t");
```

Полученные значения преобразуются в числовой формат и сохраняются в массив.

При выборе пункта «Показать массив» программа выводит на экран все элементы загруженного массива.

```
PrintArray(source, n);
```

Если сортировка уже была выполнена, дополнительно отображается отсортированный массив.

При выборе пункта «Сортировать» выполняется пузырьковая сортировка массива по возрастанию. Алгоритм последовательно сравнивает соседние элементы массива и при необходимости меняет их местами.

```
if (dst[j] > dst[j + 1])  
{  
    double tmp = dst[j];  
    dst[j] = dst[j + 1];  
    dst[j + 1] = tmp;  
}
```

Во время работы алгоритма производится подсчет количества сравнений и перестановок элементов массива.

```
(*comparisons)++;  
(*swaps)++;
```

Для повышения эффективности работы используется механизм раннего завершения сортировки. Если во время очередного прохода перестановки не выполнялись, алгоритм прекращает работу.

```
if (!swapped)  
    break;
```

Перед началом сортировки и после ее завершения фиксируется системное время, что позволяет определить продолжительность выполнения алгоритма.

```
t1 = clock();  
BubbleSort(source, sorted, n, &comparisons, &swaps);  
t2 = clock();
```

Полученные результаты выводятся на экран пользователя.

```
printf("Сравнений: %lld\n", comparisons);  
printf("Перестановок: %lld\n", swaps);  
printf("Время: %.6f сек\n", ms);
```

При выборе пункта «Сохранить результат» пользователю предлагается выбрать один из способов записи данных: перезапись исходного файла, добавление данных в конец файла или сохранение результата в новый файл.

```
printf("1. Перезаписать текущий файл\n");  
printf("2. Добавить в конец файла\n");  
printf("3. Сохранить в новый файл\n");
```

После выбора способа сохранения отсортированный массив записывается в файл.

```
WriteArray(f, sorted, n);
```

При выборе пункта «Выход» программа завершает работу.

```
return 0;
```

Подробный алгоритм работы программы и алгоритм пузырьковой сортировки представлены в разделе 4. Листинг программы приведен в приложении А.

4 Схема программ

4.1 Блок-схема программы

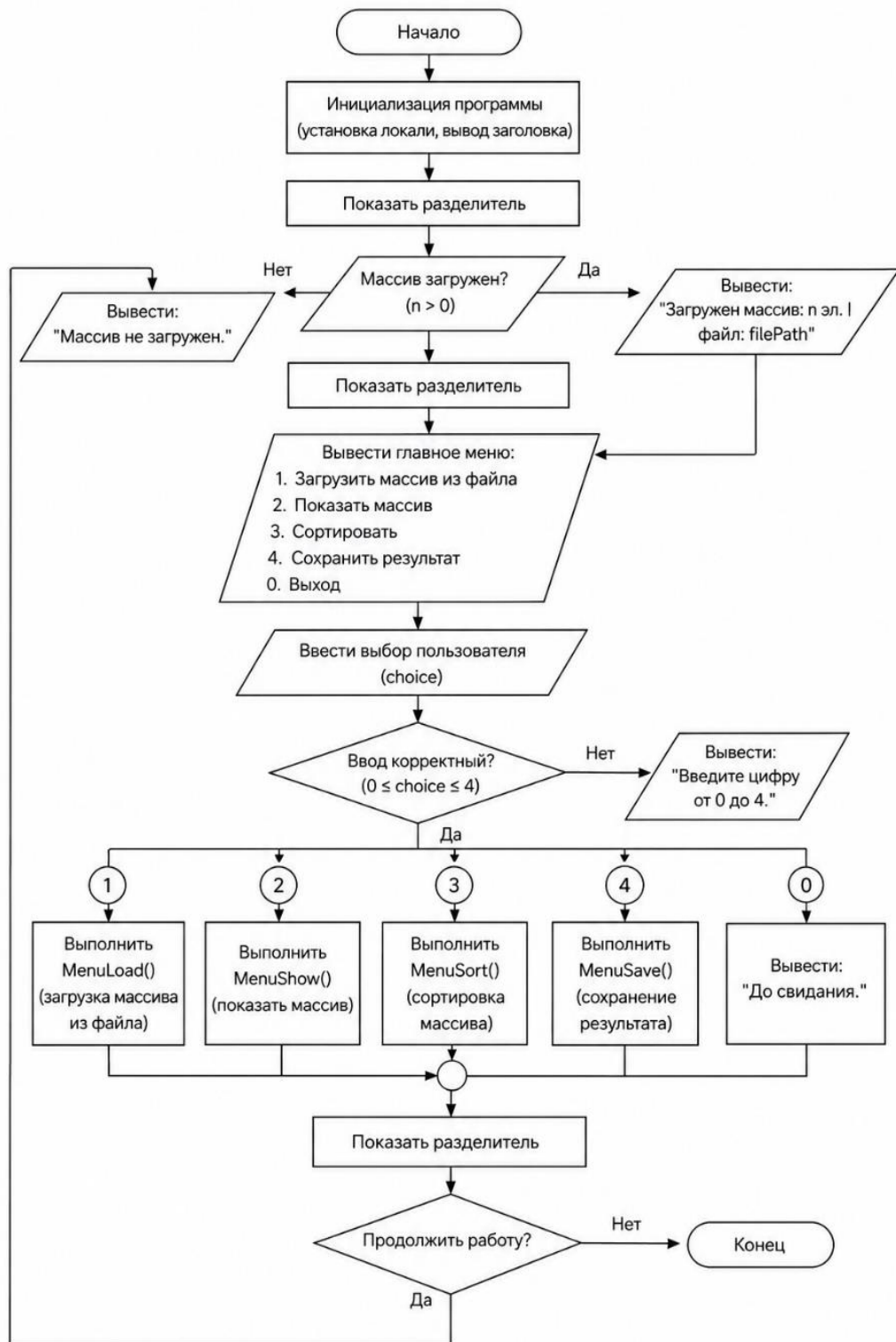


Рисунок 1 – Блок-схема программы

4.2 Блок-схема алгоритма

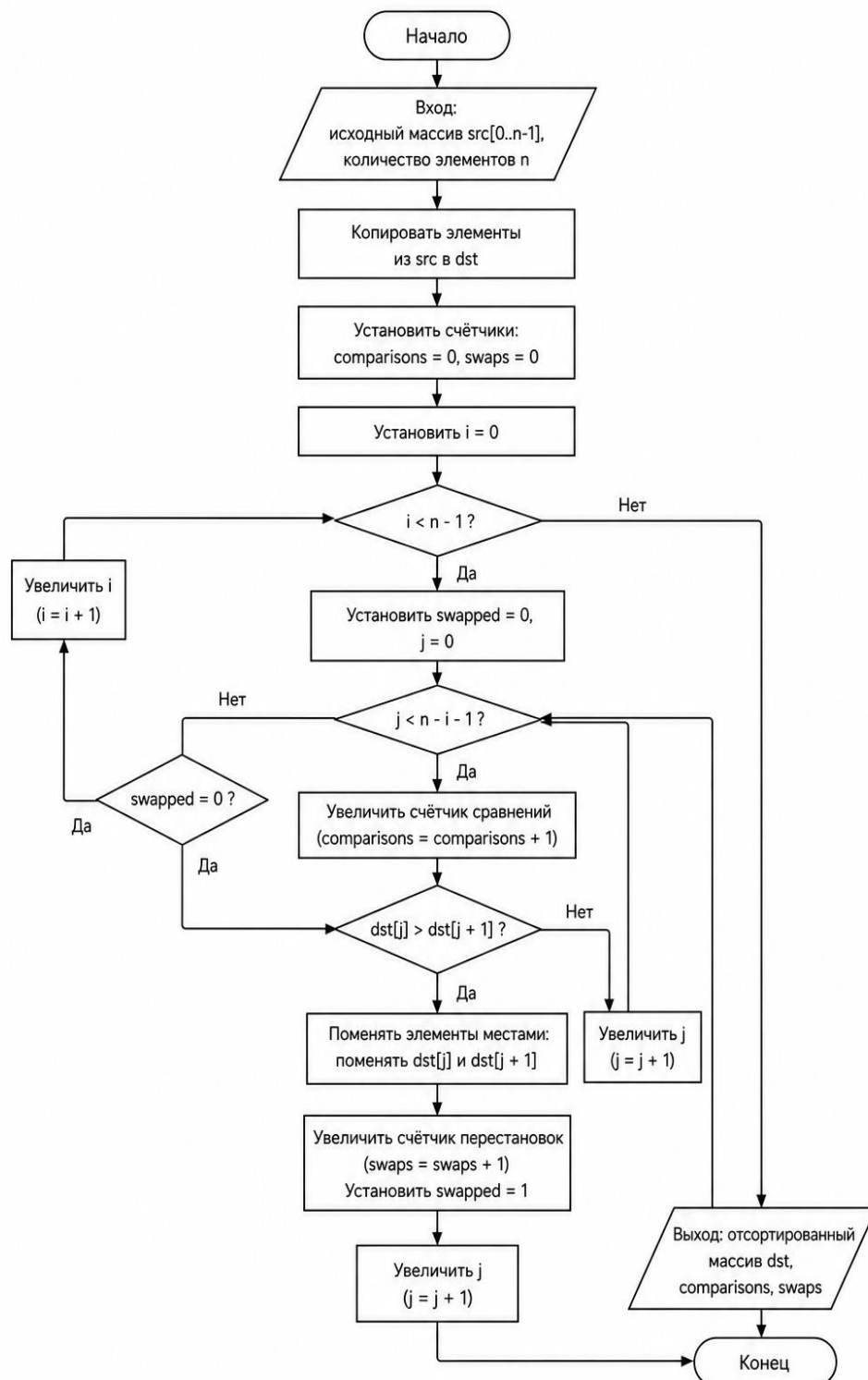


Рисунок 2 – Блок-схема алгоритма

5 Тестирование программы

Тестирование показало, что с увеличением количества элементов пропорционально увеличивается время работы программы, ниже представлен график результатов тестирования.

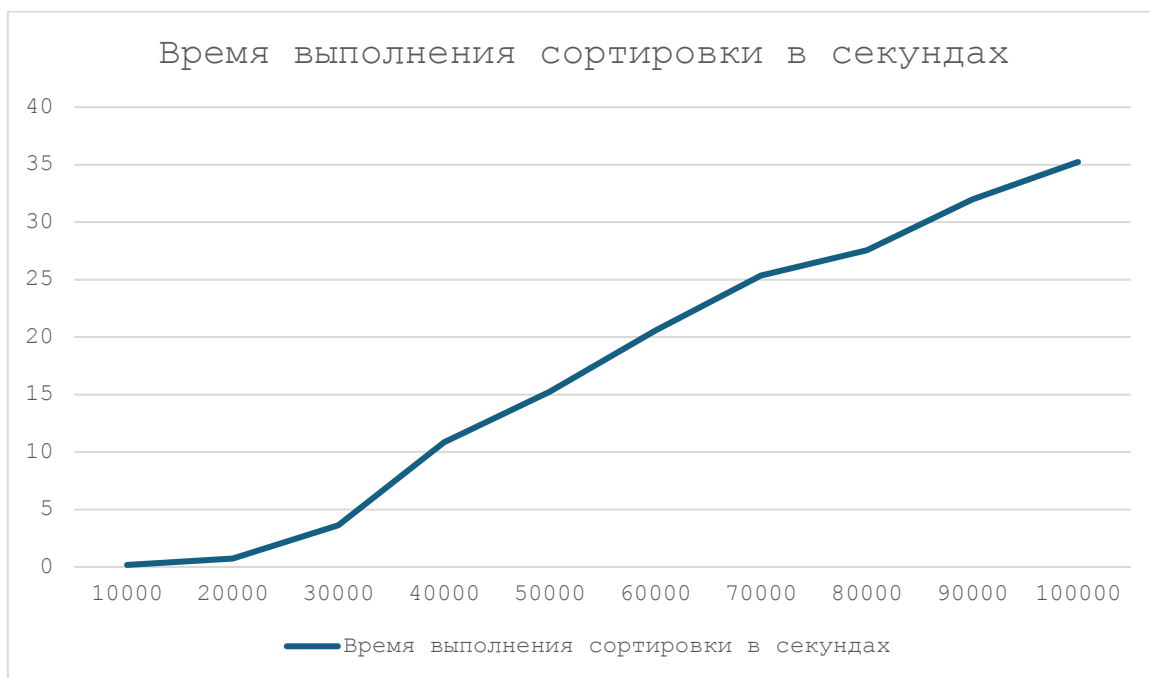


Рисунок 3 – График результатов тестирования

Тестовый набор данных, на основе которых был составлен график, представлен в таблице 1. Результаты в Приложении Б на рисунках Б.1 – Б.7

Таблица 1 – Тестовый набор данных

№	Размер массива size	Время выполнения сортировки в секундах
1	10000	0,039
2	20000	0,144
3	30000	0,33
4	40000	0,572
5	50000	0,936
6	60000	1,303
7	70000	1,779
8	80000	2,33
9	90000	2,985
10	100000	3,56

6 Отладка

Для написания и отладки программного кода была использована интегрированная среда разработки Microsoft Visual Studio, предоставляющая полный набор инструментов для разработки, компиляции и тестирования программ.

В процессе отладки применялись такие средства, как установка точек останова, пошаговое выполнение программы и контроль значений переменных. Особое внимание уделялось проверке корректности загрузки данных из файлов, работе алгоритма сортировки и сохранению результатов.

Под точкой останова понимается специальное место в программе, при достижении которого выполнение приостанавливается и управление передается отладчику. Данный инструмент позволяет анализировать текущее состояние программы, просматривать значения переменных и своевременно обнаруживать ошибки.

Отладка программы выполнялась путем последовательной проверки всех пунктов меню. Проверялась корректность открытия файлов, чтения числовых данных, обработки различных разделителей, отображения массива, выполнения пузырьковой сортировки и сохранения результатов работы программы.

Особое внимание уделялось проверке работы алгоритма пузырьковой сортировки. Контролировалось количество сравнений и перестановок элементов, а также правильность работы механизма раннего завершения сортировки при отсутствии обменов элементов массива.

Кроме того, выполнялась проверка обработки ошибочных ситуаций, связанных с вводом некорректных данных, отсутствием файлов, невозможностью открытия файлов и неправильным выбором пунктов меню. Обнаруженные в процессе разработки ошибки были устранены, что позволило обеспечить корректную и устойчивую работу программы.

Проверка работоспособности программы проводилась как в процессе разработки отдельных модулей, так и после завершения написания программного кода. В результате были устранены недочеты, связанные с обработкой файлов, пользовательским вводом и выполнением сортировки, что позволило обеспечить стабильную работу программного продукта.

7 Совместная работа

Для удобства совместной разработки был использован сервис Github Projects. Определили задачи проекта, назначили приоритет задачам.

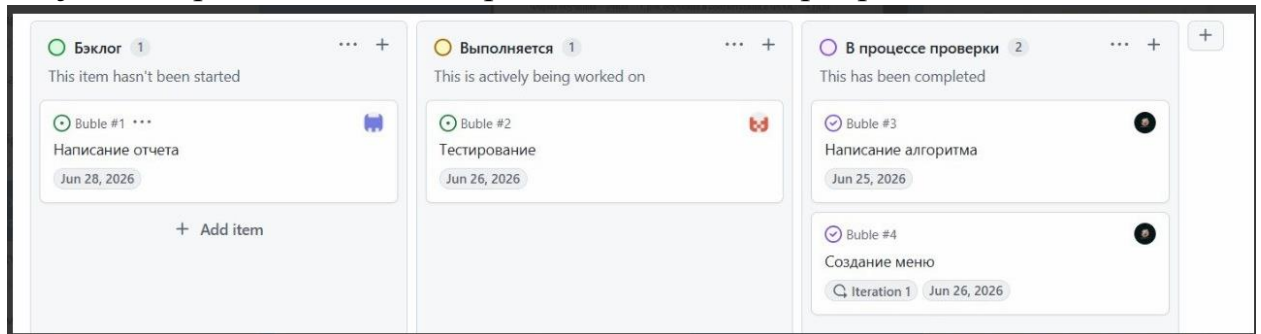


Рисунок 4 – Определние задач проекта

Распределили роли, назначили исполнителей задачам.

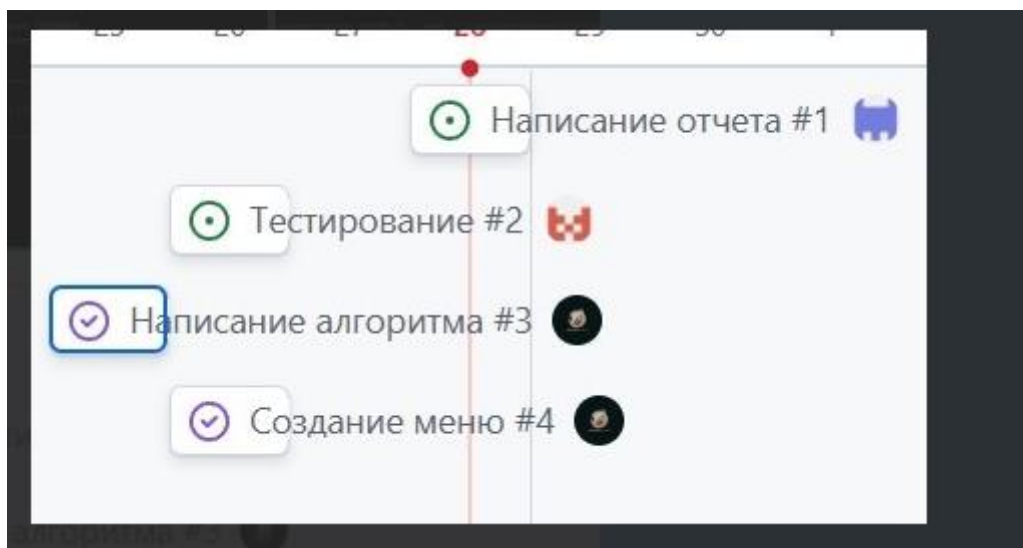


Рисунок 5 – Распределение задач проекта

Корректировали статус задач по мере выполнения.

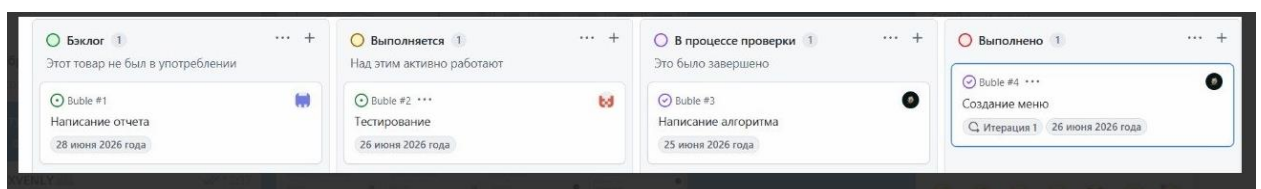


Рисунок 6 – Корректирование задач

Во время работы над выполнением задания наша бригада осуществляла работу в Github.

Мною была написана программа тестирования, она была загружена на удаленный репозиторий Github, на ветку main.

```
Илья@WIN-QDGUIKA8F7G MINGW64 ~/Desktop/bubble
$ git init
Initialized empty Git repository in C:/Users/Илья/Desktop/bubble/.git/

Илья@WIN-QDGUIKA8F7G MINGW64 ~/Desktop/bubble (master)
$ git clone https://github.com/kvertik1/Buble.git
Cloning into 'Buble'...
remote: Enumerating objects: 10, done.
remote: Counting objects: 100% (10/10), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 10 (delta 1), reused 7 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (10/10), 4.47 KiB | 4.47 MiB/s, done.
Resolving deltas: 100% (1/1), done.
```

Рисунок 7 – Проверка состояния репозитория

```
Илья@WIN-QDGUIKA8F7G MINGW64 ~/Desktop/bubble/Buble (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   Bubble.c

no changes added to commit (use "git add" and/or "git commit -a")

Илья@WIN-QDGUIKA8F7G MINGW64 ~/Desktop/bubble/Buble (main)
$ git add .
```

Рисунок 8 – Добавление файла в индекс

```
Илья@WIN-QDGUIKA8F7G MINGW64 ~/Desktop/bubble/Buble (main)
$ git commit -m "Add menu and func"
[main 81724de] Add menu and func
1 file changed, 12 insertions(+), 33 deletions(-)
```

Рисунок 9 – Фиксация изменений

```
Илья@WIN-QDGUIKA8F7G MINGW64 ~/Desktop/bubble/Buble (main)
$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 397 bytes | 397.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/kvertik1/Buble.git
   b3a2f6c..81724de  main -> main
```

Рисунок 10 – Загрузка на удаленный репозиторий

Ссылка на удаленный репозиторий:

<https://github.com/kvertik1/Buble.git>

Заключение

В ходе выполнения данной работы были изучены принципы работы алгоритма пузырьковой сортировки и особенности его применения при обработке массивов данных. Получены практические навыки разработки программ на языке Си, а также работы с текстовыми файлами и динамической обработкой данных.

В рамках работы была разработана программа, позволяющая загружать числовые данные из файлов форматов TXT и CSV, выполнять сортировку массива по возрастанию методом пузырька, определять количество сравнений и перестановок элементов, измерять время выполнения алгоритма и сохранять результаты обработки в файл.

Проведенное тестирование программы позволило оценить эффективность алгоритма на различных наборах данных. Были исследованы лучший, средний и худший случаи работы пузырьковой сортировки, что подтвердило теоретическую сложность алгоритма и показало влияние исходного порядка элементов на время выполнения сортировки.

В процессе выполнения задания были приобретены навыки коллективной разработки программного обеспечения с использованием системы контроля версий Git и сервиса GitHub. Получен опыт работы с Git Bash, создания коммитов и совместной работы над программным проектом.

В дальнейшем возможно совершенствование программы путем добавления других алгоритмов сортировки, проведения сравнительного анализа их производительности и расширения функциональных возможностей приложения.

Список используемой литературы

1. Керниган Б., Ритчи Д. Язык программирования С. — 2-е изд. — М.: Вильямс, 2015.
2. Дейтел П., Дейтел Х. Как программировать на С. — М.: Бином, 2017.
3. Вирт Н. Алгоритмы и структуры данных. — СПб.: Питер, 2018.
4. Седжвик Р. Алгоритмы на С. Фундаментальные алгоритмы и структуры данных. — М.: Вильямс, 2019.
5. Пузырьковая сортировка (Bubble Sort) [Электронный ресурс]. — Режим доступа: https://ru.wikipedia.org/wiki/Сортировка_пузырьком
6. Документация языка Си [Электронный ресурс]. — Режим доступа: <https://en.cppreference.com/w/c>

Приложение А.

Листинг программы

```
#define _CRT_SECURE_NO_WARNINGS
#include <windows.h>
#include <commdlg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <locale.h>
#include <time.h>

#pragma comment(lib, "Comdlg32.lib")

#define MAX_ARRAY 100000
#define MAX_PATH_LEN 260

double source[MAX_ARRAY]; /* исходный массив */
double sorted[MAX_ARRAY]; /* отсортированный массив */
int n = 0; /* количество элементов */
int hasSorted = 0; /* флаг: была ли сортировка */
char filePath[MAX_PATH_LEN] = "";

/* форматируем число: целое - без точки, дробное - без экспоненты */
void FormatNumber(double v, char* buf, size_t bufSize)
{
    if (v == floor(v)) {
        sprintf(buf, "%.0f", v);
    }
    else {
        char* dot;
        sprintf(buf, "%.10f", v);
        dot = strchr(buf, '.');
        if (dot) {
            char* end = buf + strlen(buf) - 1;
            while (end > dot + 1 && *end == '0') *end-- = '\0';
        }
    }
}

void PrintArray(const double* arr, int count)
{
    int i;
    char buf[64];
    for (i = 0; i < count; i++) {
        FormatNumber(arr[i], buf, sizeof(buf));
        printf("%s", buf);
        if (i < count - 1) printf(", ");
        if ((i + 1) % 10 == 0 && i < count - 1) printf("\n");
    }
    printf("\n");
}

/* режим строку на числа по разделителям */
int ParseArray(char* text)
{
    int count = 0;
```



```

char* token = strtok(text, ",; \r\n\t");

while (token) {
    char* end = NULL;
    double value;

    if (count >= MAX_ARRAY) {
        printf("Превышен лимит (%d чисел), загружено %d.\n", MAX_ARRAY,
count);
        break;
    }

    value = strtod(token, &end);
    if (end == token || *end != '\0') {
        printf("Пропущен некорректный токен: \"%s\"\n", token);
        token = strtok(NULL, ",; \r\n\t");
        continue;
    }

    source[count++] = value;
    token = strtok(NULL, ",; \r\n\t");
}

n = count;
hasSorted = 0;
return count;
}

int LoadFromFile(const char* path)
{
    FILE* f;
    long size;
    char* buf;
    size_t readCount;
    int result;

    f = fopen(path, "r");
    if (!f) {
        printf("Ошибка: не удалось открыть файл.\n");
        return 0;
    }

    fseek(f, 0, SEEK_END);
    size = ftell(f);
    rewind(f);

    buf = (char*)calloc(size + 1, 1);
    if (!buf) {
        fclose(f);
        printf("Ошибка: нехватка памяти.\n");
        return 0;
    }

    readCount = fread(buf, 1, size, f);
    fclose(f);
    buf[readCount] = '\0';

    result = ParseArray(buf);
    free(buf);
}

```

```

        return result;
    }

void WriteArray(FILE* f, const double* arr, int count)
{
    int i;
    char buf[64];
    for (i = 0; i < count; i++) {
        FormatNumber(arr[i], buf, sizeof(buf));
        if (i > 0) fprintf(f, ", ");
        fprintf(f, "%s", buf);
    }
    fprintf(f, "\n");
}

/* пузырьковая сортировка по возрастанию */
void BubbleSort(const double* src, double* dst, int count,
    long long* comparisons, long long* swaps)
{
    int i, j;
    int swapped;

    for (i = 0; i < count; i++) dst[i] = src[i];
    *comparisons = 0;
    *swaps = 0;

    for (i = 0; i < count - 1; i++) {
        swapped = 0;
        for (j = 0; j < count - i - 1; j++) {
            (*comparisons)++;
            if (dst[j] > dst[j + 1]) {
                double tmp = dst[j];
                dst[j] = dst[j + 1];
                dst[j + 1] = tmp;
                (*swaps)++;
                swapped = 1;
            }
        }
        if (!swapped) break;
    }
}

/* диалог выбора файла для открытия */
int DialogOpenFile(char* outPath)
{
    OPENFILENAMEA ofn;
    char buf[MAX_PATH_LEN] = "";

    ZeroMemory(&ofn, sizeof(ofn));
    ofn.lStructSize = sizeof(ofn);
    ofn.lpstrFilter = "Текстовые файлы (*.txt;*.csv)\0*.txt;*.csv\0Все файлы (*.*)\0*.*\0";
    ofn.lpstrFile = buf;
    ofn.nMaxFile = sizeof(buf);
    ofn.Flags = OFN_FILEMUSTEXIST | OFN_HIDEREADONLY;

    if (GetOpenFileNameA(&ofn)) {
        strcpy(outPath, buf);
        return 1;
    }
}

```

```

    }
    return 0;
}

/* диалог сохранения файла */
int DialogSaveFile(char* outPath)
{
    OPENFILENAMEA ofn;
    char buf[MAX_PATH_LEN] = "";

    ZeroMemory(&ofn, sizeof(ofn));
    ofn.lStructSize = sizeof(ofn);
    ofn.lpstrFilter = "Текстовые файлы (*.txt)\0*.txt\0Все файлы (*.*)\0*.*\0";
    ofn.lpstrFile = buf;
    ofn.nMaxFile = sizeof(buf);
    ofn.lpstrDefExt = "txt";
    ofn.Flags = OFN_OVERWRITEPROMPT;

    if (GetSaveFileNameA(&ofn)) {
        strcpy(outPath, buf);
        return 1;
    }
    return 0;
}

void PrintSep(void)
{
    printf("\n-----\n\n");
}

void MenuLoad(void)
{
    char path[MAX_PATH_LEN] = "";

    printf("Открывается диалог выбора файла...\n");
    if (!DialogOpenFile(path)) {
        printf("Файл не выбран.\n");
        return;
    }

    if (LoadFromFile(path) > 0) {
        strcpy(filePath, path);
        printf("Загружено %d чисел из \"%s\".\n", n, path);
    }
    else {
        printf("Файл не содержит корректных чисел.\n");
    }
}

void MenuShow(void)
{
    if (n == 0) {
        printf("Массив не загружен.\n");
        return;
    }

    printf("Исходный массив (%d эл.): \n", n);

```

```

    PrintArray(source, n);

    if (hasSorted) {
        printf("\nОтсортированный массив:\n");
        PrintArray(sorted, n);
    }
}

void MenuSort(void)
{
    long long comparisons, swaps;
    clock_t t1, t2;
    double ms;

    if (n == 0) {
        printf("Сначала загрузите массив (пункт 1).\n");
        return;
    }

    t1 = clock();
    BubbleSort(source, sorted, n, &comparisons, &swaps);
    t2 = clock();

    ms = (double)(t2 - t1) / CLOCKS_PER_SEC;
    hasSorted = 1;

    printf("Сортировка завершена.\n");
    PrintSep();
    printf("Элементов:      %d\n", n);
    printf("Сравнений:      %lld\n", comparisons);
    printf("Перестановок:    %lld\n", swaps);
    printf("Время:          %.6f сек\n", ms);
    PrintSep();
    printf("Нажмите Enter для просмотра отсортированного массива...\n");
    getchar();
    PrintSep();
    printf("Отсортированный массив:\n");
    PrintArray(sorted, n);
    PrintSep();
}

void MenuSave(void)
{
    int choice;
    char path[MAX_PATH_LEN];
    FILE* f;

    if (!hasSorted) {
        printf("Сначала выполните сортировку (пункт 3).\n");
        return;
    }

    printf("Куда сохранить?\n");
    printf(" 1. Перезаписать текущий файл (%s)\n",
        filePath[0] ? filePath : "не задан");
    printf(" 2. Добавить в конец текущего файла\n");
    printf(" 3. Сохранить в новый файл\n");
    printf("Ваш выбор: ");

```

```

    if (scanf("%d", &choice) != 1) {
        while (getchar() != '\n');
        printf("Некорректный ввод.\n");
        return;
    }
    while (getchar() != '\n');

    /* если текущий файл не задан — идём в диалог */
    if ((choice == 1 || choice == 2) && filePath[0] == '\0') {
        printf("Текущий файл не задан, открывается диалог сохранения...\n");
        choice = 3;
    }

    if (choice == 3) {
        printf("Открывается диалог сохранения...\n");
        if (!DialogSaveFile(path)) {
            printf("Файл не выбран.\n");
            return;
        }
    }
    else {
        strcpy(path, filePath);
    }

    if (choice == 2) {
        f = fopen(path, "a");
        if (!f) { printf("Ошибка открытия файла.\n"); return; }
        fprintf(f, "\n-----\n");
        WriteArray(f, sorted, n);
        fclose(f);
    }
    else {
        f = fopen(path, "w");
        if (!f) { printf("Ошибка открытия файла.\n"); return; }
        WriteArray(f, sorted, n);
        fclose(f);
    }

    printf("Сохранено в \"%s\".\n", path);
}

int main(void)
{
    int choice;

    setlocale(LC_ALL, "Russian");

    printf("=====\n");
    printf("        Пузырьковая сортировка (Си)\n");
    printf("=====\n");

    for (;;) {
        printf("\n");
        PrintSep();
        if (n > 0)
            printf("Загружен массив: %d эл. | файл: %s\n", n, filePath);
        else
            printf("Массив не загружен.\n");
        PrintSep();
    }
}

```

```

printf("1. Загрузить массив из файла\n");
printf("2. Показать массив\n");
printf("3. Сортировать\n");
printf("4. Сохранить результат\n");
printf("0. Выход\n");
printf("Ваш выбор: ");

if (scanf("%d", &choice) != 1) {
    while (getchar() != '\n');
    printf("Введите цифру от 0 до 4.\n");
    continue;
}
while (getchar() != '\n');

printf("\n");

switch (choice) {
case 1: MenuLoad(); break;
case 2: MenuShow(); break;
case 3: MenuSort(); break;
case 4: MenuSave(); break;
case 0:
    printf("До свидания.\n");
    return 0;
default:
    printf("Неверный пункт. Введите от 0 до 4.\n");
}
}
}

```

Приложение Б. Результаты тестирования программы

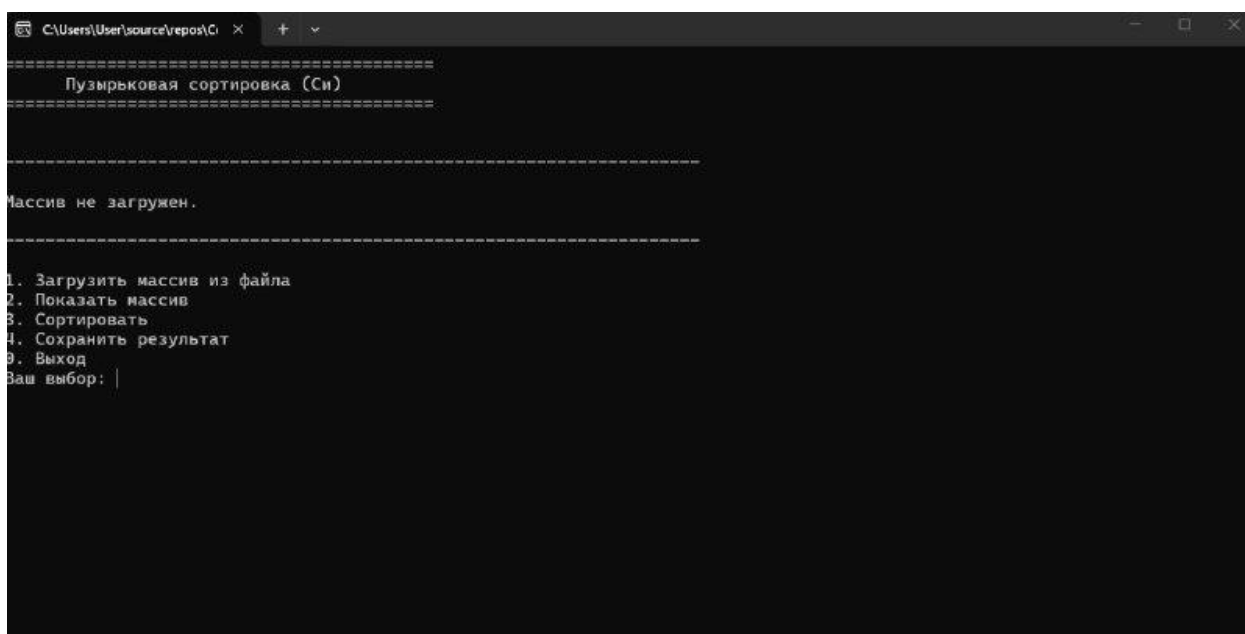


Рисунок Б.1

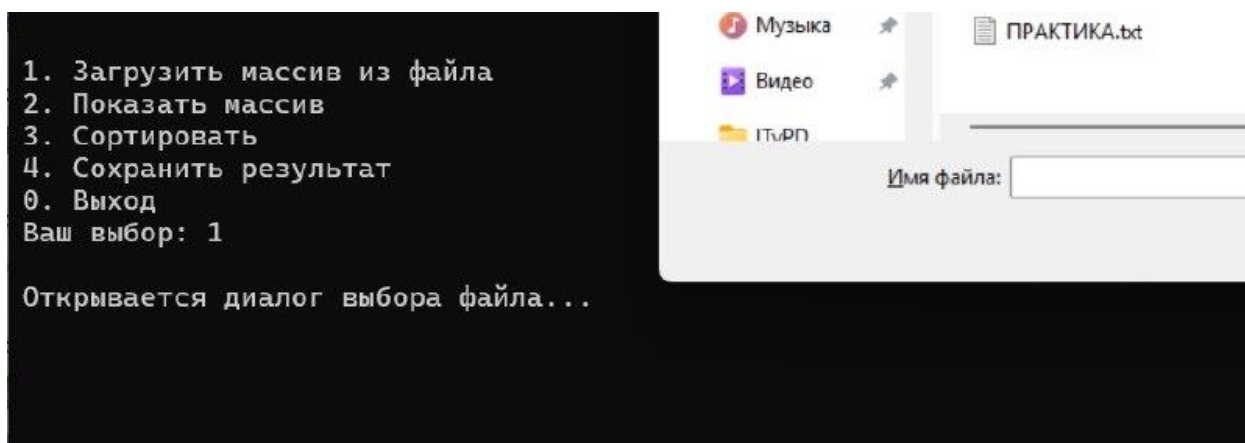


Рисунок Б.2

```
Открывается диалог выбора файла...
Превышен лимит (100000 чисел), загружено 100000.
Загружено 100000 чисел из "C:\Users\User\Desktop\ПРАКТИКА.txt".

-----

Загружен массив: 100000 эл. | файл: C:\Users\User\Desktop\ПРАКТИКА.txt
-----

1. Загрузить массив из файла
2. Показать массив
3. Сортировать
4. Сохранить результат
0. Выход
Ваш выбор: |
```

Рисунок Б.3

```
Загружен массив: 100000 эл. | файл: C:\Users\User\Desktop\ПРАКТИКА
-----

1. Загрузить массив из файла
2. Показать массив
3. Сортировать
4. Сохранить результат
0. Выход
Ваш выбор: 3

Сортировка завершена.

-----

Элементов:      100000
Сравнений:      4999805009
Перестановок:   2500092300
Время:         35,173000 сек
-----

Нажмите Enter для просмотра отсортированного массива...
|
```

Рисунок Б.4